Photo Courtesy - Sunny's Paw

Applied AI System -Pet Service Scheduler

BY- Priyasri Sankaran

Codepath-AI Foundation Engineering

April, 2026

Pet Service scheduler

- Intelligent system that generates safe, personalized daily care plans for pets
- Upgraded from a simple rule-based scheduler to a full applied AI pipeline

System function

- Produces validated daily pet care schedulers
- Uses pet profile + tasks + owner priorities
- Ensures safety through rule retrieval and guardrails
- Adapts to health conditions, breed limits, and age-specific needs

Original project

- Simple prioritization of tasks(feed, groom, walk)
- No knowledge base retrieval
- No LLM reasoning
- No safety validation
- Could not handle nuanced or special cases

New AI features added

- Retrieval- Augmented Generation(RAG)

  Pulls relevant rules from a curated knowledge base

  Keeps system reliable and updatable
- Agentic Workflow

  Analyze → Retrieve → Plan →Evaluate →Decide
- Deterministic Guardrails

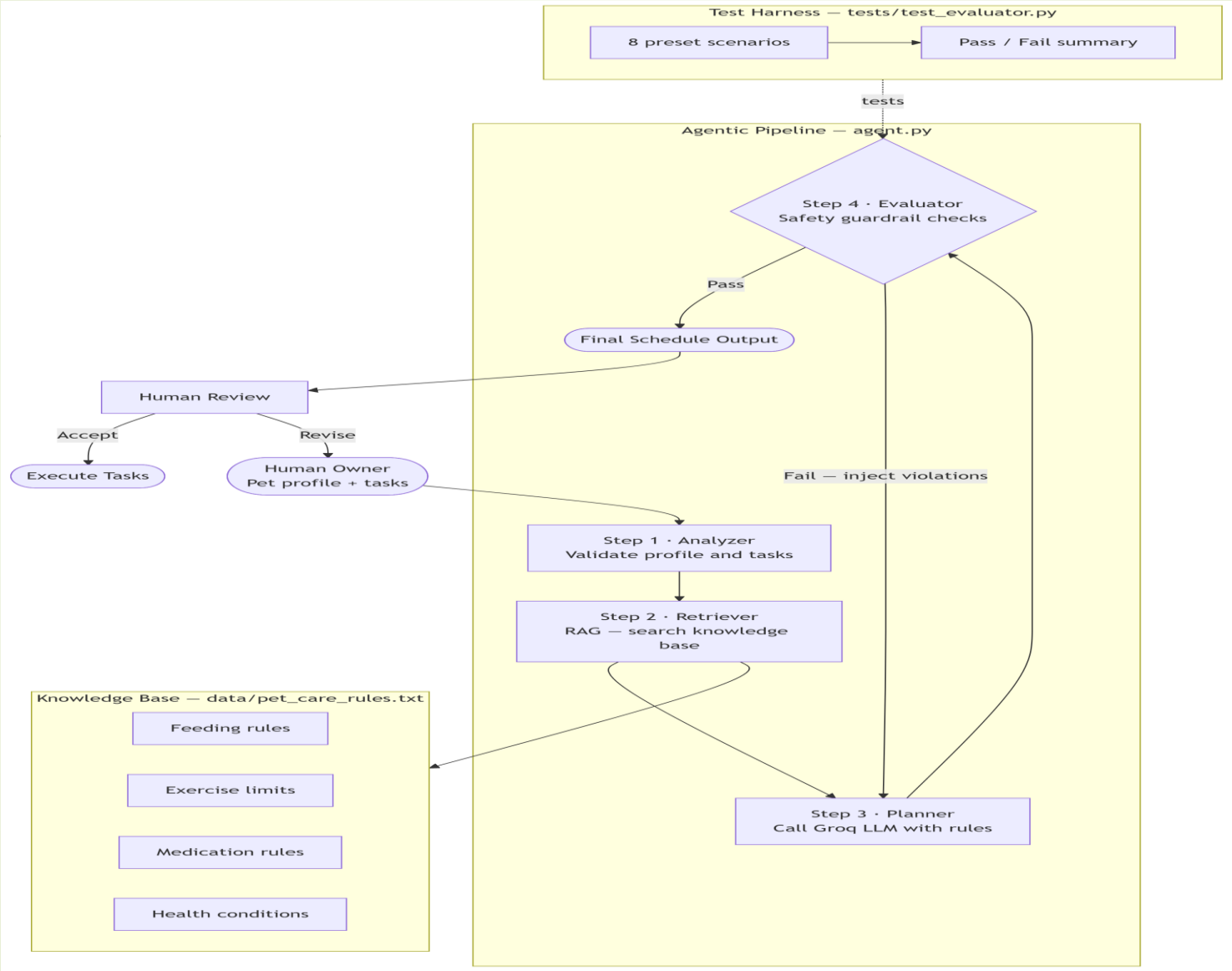
  Enforces safety constrains before outputting schedule

RAG Module

- Veterinary-informed rules in a text file
- Covers feeding-exercise spacing. Medical conditions, timing, breed limits, senior care
- Retriever provides only relevant rules into LLM prompt
- No retraining needed to update rules

Agentic Workflow

- Analyze: Validate pet profile + tasks
- Retrieve: Fetch relevant safety rules
- Plan: LLM generates schedule using structured prompt
- Evaluate: Determine guardrails checks for violations
- Re-plan: If unsafe schedules from reaching user


Test Harness — tests/test_evaluator.py
8 preset scenarios
Pass / Fail summary
tests
Agentic Pipeline — agent.py
Step 4 · Evaluator
Safety guardrail checks
Pass
Final Schedule Output
Human Review
Accept
Revise
Execute Tasks
Human Owner
Pet profile + tasks
Fail — inject violations
Step 1 · Analyzer
Validate profile and tasks
Step 2 · Retriever
RAG — search knowledge base
Knowledge Base — data/pet_care_rules.txt
Feeding rules
Exercise limits
Medication rules
Health conditions
Step 3 · Planner
Call Groq LLM with rules

Example Outputs

- **Bella (Healthy Dog):** All checks passed; 19 rules applied
- **Max (Heart Condition):** Walk duration reduced; 16 rules applied
- **Whiskers (Senior Cat):** Frequent small meals; arthritis-safe activities; 14 rules applied

Design Decisions

- Plain text RAG file → simple, transparent
- Deterministic evaluator → reliable safety
- Multi-step pipeline → modular + testable
- Re-planning loop → better user experience
- Owner priorities injected directly into prompt

Testing Summary

- Unit tests for parsing, retrieval, guardrails
- Integration tests for full pipeline
- Edge case handling (conflicting priorities, unknown species)
- Deterministic evaluator made tests highly reliable

- Reliable AI requires scaffolding around the model
- Guardrails define correctness early
- Modular design simplifies debugging
- Prompt engineering is iterative and structured

Future Work

- Expand rule sets for more species
- Replace keyword retrieval with vector search
- Add Streamlit UI
- Support multi-day scheduling